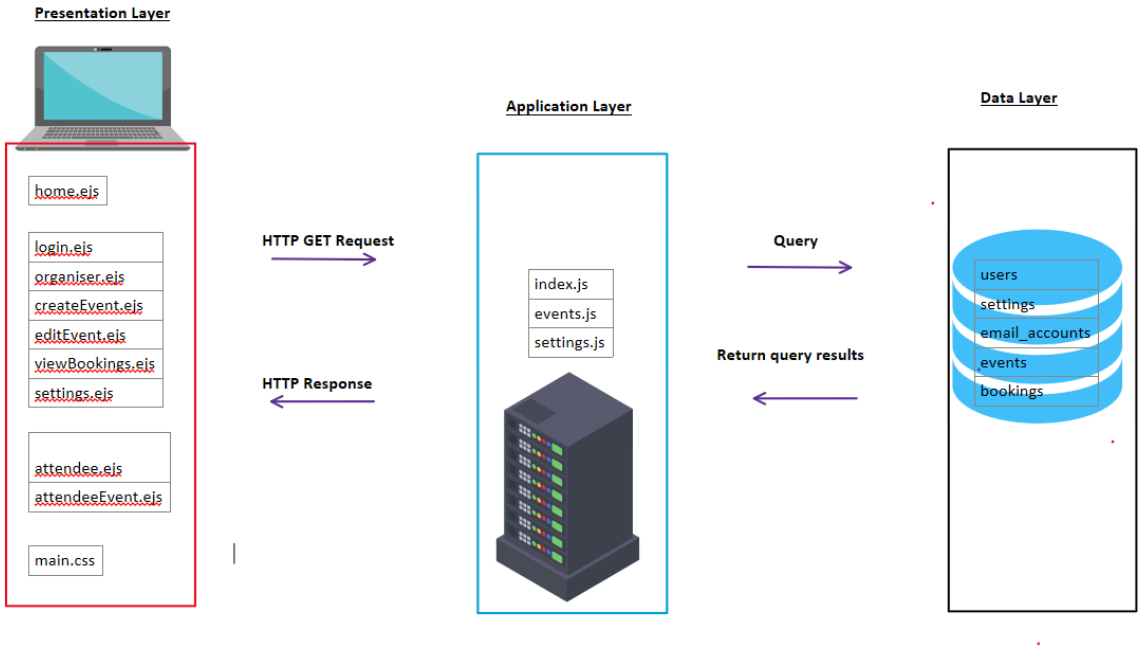
VoxPulse Project Report



This project is built on a three tier architecture.

Presentation Layer

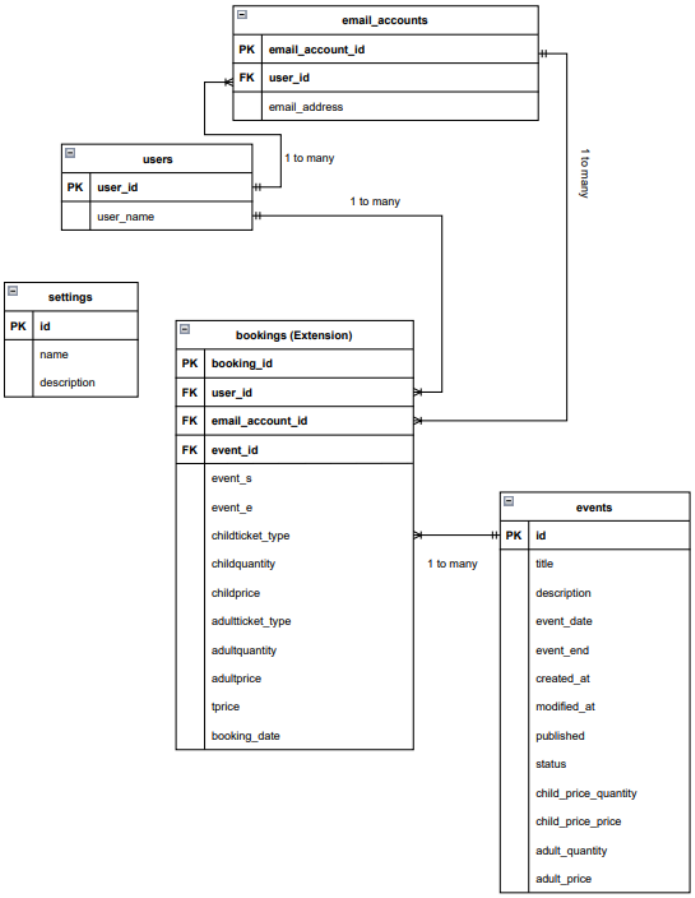
Users interact with dynamically rendered EJS pages in their browser where users land on a home page, organisers log in to manage events, and attendees browse and book tickets. Static CSS and Bootstrap to provide a consistent look and feel across all views.

Application Layer

Utilises Node.js and Express. Express routes handle HTTP GET requests to render pages like viewing events or the organiser portal and POST requests to process forms like creating an event or submitting a booking. Each route implements business logic for validation, session checks, and more and then reads from or writes to the database before calling res.render() for GET or redirecting for POST.

Data Layer

Data stored comprises 5 tables. The extension introduces one new table, bookings, and a corresponding Express route GET /organiser/events/view-bookings and a single EJS template to display every booking. When organisers comes to that endpoint, the server issues a SELECT * FROM bookings query, receives the results, and returns the rendered bookings page.



This is an entity relationship diagram of the relational database schema for Vox Pulse booking system, showing tables like users, email_accounts, events, bookings, and settings with their columns, primary keys, and foreign-key links.

users: stores each person's name.

email_accounts: lets a user save multiple email addresses where each entry links back to the owner in users.

events: defines each event's details like date, ticket types (adult/child), description and more..

Bookings (extension): The main table where whenever someone reserves tickets, it ties together the user, specific email_account they chose, and the event and other details.

settings: stores the site name and description used throughout the application.

Extension Description

Extension Implementation

For my project, I implemented a View Bookings page designed specifically for the organiser page . This extension allows organisers to view all attendee bookings, and specifically filter them by event, and inspect detailed ticket breakdowns including child and adult ticket quantities, total prices, and booking dates. The data comes from the Attendee events page where the user keys in the information and books the ticket which is stored in the database and called to view the booking.I used client side filtering with JavaScript to make this interaction fast and seamless. Each booking card shows key details such as the event title, attendee name, and total price. If the organiser wants to dive deeper, they can expand an accordion component to see a detailed breakdown of adult and child ticket quantities and prices.

I used Bootstrap components to keep the design responsive and user friendly. For the backend, I structured my code using modular helper functions, separating logics like inserting users or sending emails into their own functions which helped to keep my code maintainable and readable.

To ensure the system is reliable, I included server side validation so tickets can't be oversold and automatic database updates for ticket quantities after each successful booking.

Pay attention to

I want you to pay close attention to the Get route view-booking.This function fetches all booking records and joins them with related user, email, and event details to show a complete booking overview for the organiser.The LEFT JOINs link the bookings table to users, email_accounts, and events to display full relevant details allowing the organiser to view complete booking data in one query.

What Was Done to Implement It

This extension involves backend enhancements using Node.js and Express, particularly within index.js and events.js. In index.js, the core booking logic handles attendee requests by checking if a user exists, inserting email records, fetching event data, and storing the booking. These steps are cleanly separated using helper functions like insertUser, insertEmail, getEvent, and insertBooking, making the flow modular and maintainable. It also updates ticket availability to prevent overbooking, enforcing business rules in the backend.

In events.js, the /view-bookings route fetches and prepares all booking records for organisers. It compiles attendee and event details, then renders them in viewBooking.ejs as Bootstrap cards with expandable ticket sections along with a dropdown filter which enables real time event filtering without page reloads.

Database Support

The extension relies on four database tables. The events table includes ticket related fields like child_price_quantity, adult_quantity, and their respective prices, allowing event specific pricing and availability. The users and email_accounts tables store attendee information and help prevent data duplication. The bookings table is the main table that connects users, emails, and events and stores the quantities, ticket types, prices, and timestamps for each booking.

The system is designed to keep everything connected and accurate. When someone makes a booking, ticket quantities are automatically updated so organisers always see the latest availability.This helps keep the data clean and up to date without any extra work.

Key Implementation Snippets

View Bookings Route - event.js lines 198 - 244

```
router.get('/view-bookings', (req, res, next) => {
  global.db.all(
    `SELECT b.booking_id,
            u.user_name,
            ea.email_address,
            e.title,
            b.event_s, b.event_e,
            b.childticket_type,
            b.childquantity,
            e.child_price_price,
            b.adultticket_type,
            b.adultquantity,
            e.adult_price,
            b.tprice,
            b.booking_date
     FROM bookings b
     LEFT JOIN users u       ON b.user_id = u.user_id
     LEFT JOIN email_accounts ea ON b.email_account_id = ea.email_account_id
     LEFT JOIN events e      ON b.event_id = e.id
     ORDER BY b.booking_date DESC`,
    (err, bookings) => {
      if (err) return next(err);

      // Convert the raw date strings into human-friendly formats
      bookings.forEach(booking => {
        booking.start_date   = new Date(booking.event_s).toLocaleDateString();
        booking.end_date     = new Date(booking.event_e).toLocaleDateString();
        booking.booking_date = new Date(booking.booking_date).toLocaleString();
      });
```

```
    // Get the site's name & description for the page header
    global.db.get(
      'SELECT name, description FROM settings WHERE id = 1',
      (err, site) => {
        if (err) return next(err);

        // Render the template with all gathered data
        res.render('viewBookings', {
          bookings,
          name:        site?.name        || 'Untitled Site',
          description: site?.description || 'No description available'
        });
      }
    );
  }
);
});
```

Fetch and join data: Runs a SQL query to retrieve all bookings, along with each booking's user name, email address, and event details, ordered by newest first.

Format dates: Converts the raw start, end, and booking timestamps into readable date strings.

Load site settings: Retrieves the site's name and description from the settings table.

Render page: Passes the enriched bookings array plus site title and tagline into the viewBookings template for display.

Get route for attendeeEvents page - index.js lines 206 to 230

```
// Route to display event details for an attendee
app.get('/attendee/events/:id', (req, res, next) => {
  const eventId = req.params.id;  // Get event ID from URL

  // Get site name and description
  global.db.get('SELECT name, description FROM settings WHERE id = 1', (err, site) => {
    if (err) return next(err);

    // Get the event details from the database
    global.db.get(
      `SELECT * FROM events WHERE id = ? AND status = 'published'`,
      [eventId],
      (errEvent, event) => {
        if (errEvent) return next(errEvent);
        if (!event) return res.status(404).send('Event not found');

        // Render the event details page for the attendee event page
        res.render('attendeeEvent', {
          name: site?.name || 'Untitled Site',
          description: site?.description || '',
          event  // The updated event data is passed here
        });
      }
    );
  });
});
```

Extract event ID: Reads the :id parameter from the URL to know which event to load.

Load site settings: Queries the settings table for the site name and description (row id = 1).

Get event record: Retrieves the event with that ID only if its status is published, returning a 404 if not found.

Render attendee page: Passes the site information and event data into the attendeeEvent template for display.

viewBooking post route and helper functions index.js lines

```
app.post('/attendee/events/:id/book', (req, res, next) => {
  const eventId = req.params.id; // Get event ID from URL
  const { child_price_quantity, adult_quantity, attendee_name, attendee_email } = req.body;

  // Check if user already exists
  global.db.get('SELECT user_id FROM users WHERE user_name = ?', [attendee_name], (err, user) => {
    if (err) return next(err);


    // For New Users
    // If the user does not exist, create a new user
    if (!user) {
      insertUser(attendee_name, (err, userId) => {
        if (err) return next(err);

        // After user creation, insert email
        insertEmail(attendee_email, userId, (err, emailAccountId) => {
          if (err) return next(err);

          // Get the event details
          getEvent(eventId, (err, event) => {
            if (err) return next(err);
            if (!event) return res.status(404).send('Event not found');

            // Calculate total price for booking
            const childPriceTotal = child_price_quantity * event.child_price_price;
            const adultTotal = adult_quantity * event.adult_price;
            const totalPrice = childPriceTotal + adultTotal;
```

```
// Update ticket quantities in the events table
const newChildQuantity = event.child_price_quantity - child_price_quantity;
const newAdultQuantity = event.adult_quantity - adult_quantity;

// Ensure there are enough tickets
if (newChildQuantity < 0 || newAdultQuantity < 0) {
  return res.status(400).send('Not enough tickets available');
}

// Insert booking details
insertBooking(userId, emailAccountId,
  eventId, child_price_quantity, adult_quantity, totalPrice, event, (err) => {
  if (err) return next(err);

  // Update the event with the new ticket quantities
  global.db.run(
    `UPDATE events
     SET child_price_quantity = ?, adult_quantity = ?
     WHERE id = ?`,
    [newChildQuantity, newAdultQuantity, eventId],
    (errUpdate) => {
      if (errUpdate) return next(errUpdate);
      res.redirect('/attendee');  // Redirect to attendee page after successful booking
    }
```

```
} else {

  // For Existing Users
  // If user exists, insert email and proceed with booking
  insertEmail(attendee_email, user.user_id, (err, emailAccountId) => {
    if (err) return next(err);

    // Get the event details
    getEvent(eventId, (err, event) => {
      if (err) return next(err);
      if (!event) return res.status(404).send('Event not found');

      // Calculate total price for booking
      const childPriceTotal = child_price_quantity * event.child_price_price;
      const adultTotal = adult_quantity * event.adult_price;
      const totalPrice = childPriceTotal + adultTotal;

      // Update ticket quantities in the events table
      const newChildQuantity = event.child_price_quantity - child_price_quantity;
      const newAdultQuantity = event.adult_quantity - adult_quantity;

      // Ensure there are enough tickets
      if (newChildQuantity < 0 || newAdultQuantity < 0) {
        return res.status(400).send('Not enough tickets available');
      }
```

```
// Insert booking details
insertBooking(user.user_id, emailAccountId,
  eventId, child_price_quantity, adult_quantity, totalPrice, event, (err) => {
  if (err) return next(err);

  // Update the event with the new ticket quantities
  global.db.run(
    `UPDATE events
     SET child_price_quantity = ?, adult_quantity = ?
     WHERE id = ?`,
    [newChildQuantity, newAdultQuantity, eventId],
    (errUpdate) => {
      if (errUpdate) return next(errUpdate);
      res.redirect('/attendee');  // Redirect to attendee page after successful booking
    }
```

Parse request: Gets event ID and booking inputs from the URL and form.

Ensure user & email: Checks if the attendee exists, creates them and their email record if new, or just the email if returning.

Validate & compute: Loads the event, ensures enough tickets remain, and calculates the total cost.

Finalize booking: Inserts the booking, decrements available ticket counts, and redirects back to the attendee page.

TOTAL WORD COUNT : 1036 , the extra 36 come from some titles and sub titles for the report